Event Ticketing & Verification System – Requirements and Google AI Build Guide

Prepared for: Virginia Njeri
Target tool: Google AI Studio (Build mode) – https://aistudio.google.com/apps
Alternative: Google Antigravity (desktop agentic IDE) – https://antigravity.google/download

1. Which Google AI tool to use

Tool	Best for	Can you download the code?
Google AI Studio – Build mode	Fastest start in the browser. Generates a full-stack web app (React frontend + Node.js server).	Yes – "Download App" exports a ZIP; you can also sync to GitHub or deploy to Cloud Run.
Google Antigravity	Larger, production-grade builds on your own computer, with agents that plan, code, test and verify.	Code is created directly in a folder on your machine.
Firebase Studio	Browser IDE with Firebase integration.	Yes – "Zip and Download" or export to GitHub.

Recommended path: build the first version in Google AI Studio Build mode, download the ZIP (or push to GitHub), then continue in Antigravity or VS Code for production hardening.

Note: when you run the downloaded ZIP elsewhere, set the `GEMINI_API_KEY` environment variable (only needed if the app uses Gemini features).

Sources:

- Google AI Studio Build mode docs: https://aistudio.google.com/docs/aistudio-build-mode
- Firebase Studio download FAQ: https://firebase.google.com/docs/studio/troubleshooting
- Google Antigravity: https://antigravity.google/

2. System overview

A web platform where organizers create events and sell tickets. Buyers pay (M-Pesa or card) and receive QR-code tickets by email/SMS. At the venue, gate staff scan the QR codes on their phones. The system confirms each ticket in real time and blocks duplicate or fake tickets.

3. Users of the system (roles)

3.1 Super Admin (platform owner)

- Approve, suspend or delete organizer accounts
- View all events, sales, and platform revenue
- Set platform commission/service fee
- Manage refunds and disputes
- View audit logs (who did what and when)
- Manage system settings (payment keys, email/SMS settings)

3.2 Event Organizer

- Register and manage organization profile
- Create, edit, publish, unpublish and cancel events
- Define ticket types (e.g. Early Bird, Regular, VIP, VVIP, Group) with price, quantity, sale start/end dates
- Create discount/promo codes
- Add gate staff and assign them to specific events or gates
- View real-time sales, revenue and check-in dashboards
- Export attendee lists (CSV/Excel)
- Issue complimentary tickets
- Request payouts

3.3 Gate Staff / Verifier

- Log in on a phone (mobile-friendly web page)
- See only the events assigned to them
- Scan ticket QR codes using the phone camera
- Manually search a ticket by ticket number, phone or email (fallback)
- See instant result: VALID (green), ALREADY USED (orange, with time and gate of first scan), INVALID (red)
- Work in offline mode with sync when connection returns
- See their own scan count and history

3.4 Attendee / Buyer

- Browse and search public events (by date, location, category)
- Buy tickets with or without an account (guest checkout)
- Pay with M-Pesa (STK Push) or card
- Receive tickets by email and SMS (QR code + PDF)
- View "My Tickets" and download them again

- Transfer a ticket to another person
- Request a refund (if the organizer allows it)

3.5 Guest visitor (not logged in)

- View public event pages and ticket prices
- Start the checkout process

4. Functional requirements

4.1 Authentication & accounts

- FR1: Sign up / log in with email + password and Google sign-in
- FR2: Role-based access control (Super Admin, Organizer, Gate Staff, Attendee)
- FR3: Password reset by email
- FR4: Phone number verification by OTP (optional)
- FR5: Session timeout and secure logout

4.2 Event management

- FR6: Create an event with title, description, category, banner image, venue, map location, start/end date & time, and capacity
- FR7: Save as draft, publish, unpublish, cancel
- FR8: Multiple ticket types per event, each with price, quantity, sale period and per-order limit
- FR9: Promo codes (percentage or fixed amount, usage limit, expiry)
- FR10: Public event page with a shareable link

4.3 Ticket purchase

- FR11: Select ticket type and quantity; real-time availability check
- FR12: Hold tickets for 10 minutes during checkout to prevent overselling
- FR13: Collect buyer name, email and phone
- FR14: Payment via M-Pesa Daraja API (STK Push) and card (Paystack/Stripe)
- FR15: Confirm payment through a callback/webhook before issuing tickets
- FR16: Generate a unique ticket per seat/person with a ticket number and QR code
- FR17: Send tickets by email (PDF attachment) and SMS (link)
- FR18: Order receipt / invoice

4.4 Ticket security

- FR19: The QR code contains a signed token (HMAC/JWT) – not just the ticket ID – so it cannot be forged
- FR20: Each ticket has a status: ACTIVE, USED, CANCELLED, REFUNDED, TRANSFERRED
- FR21: Ticket transfer invalidates the old QR code and issues a new one
- FR22: Optional: QR rotates every 30 seconds in the "My Tickets" view, to stop screenshot sharing

4.5 Verification / check-in

- FR23: Camera-based QR scanner in the browser
- FR24: Server checks signature, event, ticket status and gate permissions
- FR25: Mark ticket as USED atomically (a database transaction) so two gates cannot accept the same ticket
- FR26: Show the result with colour, sound and vibration
- FR27: Record every scan (ticket, staff, gate, time, result)
- FR28: Offline mode: download the event's valid ticket list (hashed), verify locally, sync later and flag conflicts
- FR29: Manual search check-in
- FR30: Support re-entry (optional setting: allow exit and re-entry)

4.6 Dashboards & reports

- FR31: Organizer dashboard – tickets sold per type, revenue, check-ins vs sold, sales over time chart
- FR32: Live check-in counter during the event
- FR33: Export attendees and scan logs to CSV/Excel
- FR34: Super Admin dashboard – all organizers, events, total revenue, commission

4.7 Payments, refunds & payouts

- FR35: Record every transaction (M-Pesa receipt number, amount, status)
- FR36: Refunds that cancel the related tickets automatically
- FR37: Organizer payout requests and Super Admin approval

4.8 Notifications

- FR38: Email – order confirmation, tickets, event reminders (24 h before), event changes/cancellation
- FR39: SMS – ticket link and reminders (e.g. Africa's Talking)

4.9 AI features (Gemini)

- FR40: AI generates an event description from a few keywords
- FR41: AI helper chat for attendees ("What time does the gate open?")
- FR42: AI sales insights for organizers ("Which ticket type is selling best?")
- FR43: Fraud alert – flag unusual scan patterns (e.g. one ticket scanned at two gates within seconds)

5. Non-functional requirements

- NFR1 Performance: ticket verification responds in under 1 second
- NFR2 Scalability: handle at least 1,000 concurrent buyers during ticket launches
- NFR3 Security: HTTPS everywhere, hashed passwords, signed QR tokens, payment keys only on the server, rate limiting on login and scan endpoints
- NFR4 Availability: 99.5% uptime during events
- NFR5 Usability: mobile-first design; scanner usable with one hand
- NFR6 Data protection: comply with the Kenya Data Protection Act 2019 (consent, data minimisation, right to deletion)
- NFR7 Auditability: every admin, organizer and scan action logged
- NFR8 Backup: daily database backups

6. Suggested tech stack

- Frontend: React + Tailwind CSS (the default in AI Studio Build mode)
- Backend: Node.js + Express (server runtime in AI Studio)
- Database: Firebase Firestore (or PostgreSQL)
- Authentication: Firebase Authentication
- QR generation: `qrcode` npm package; QR scanning: `html5-qrcode`
- PDF tickets: `pdfkit`
- Payments: Safaricom M-Pesa Daraja API (STK Push), Paystack for cards
- Email: Resend / SendGrid / Nodemailer
- SMS: Africa's Talking
- AI: Gemini API
- Hosting: Google Cloud Run or Firebase Hosting

7. Database (main collections/tables)

- users (id, name, email, phone, role, status, createdAt)
- organizations (id, ownerId, name, logo, payoutDetails, status)
- events (id, orgId, title, description, category, venue, location, startAt, endAt, capacity, status, bannerUrl)
- ticketTypes (id, eventId, name, price, quantity, sold, saleStart, saleEnd, maxPerOrder)
- promoCodes (id, eventId, code, type, value, usageLimit, used, expiresAt)
- orders (id, buyerId, eventId, items, subtotal, discount, total, status, paymentMethod, createdAt)
- payments (id, orderId, provider, receiptNo, amount, status, rawCallback)
- tickets (id, orderId, eventId, ticketTypeId, holderName, holderEmail, ticketNumber, qrToken, status, checkedInAt, checkedInBy, gate)
- staffAssignments (id, eventId, staffUserId, gate)
- scanLogs (id, ticketId, eventId, staffId, gate, result, timestamp, deviceId)
- refunds (id, orderId, amount, reason, status)
- payouts (id, orgId, amount, status, requestedAt, approvedBy)
- auditLogs (id, userId, action, entity, entityId, timestamp)

8. Main pages/screens

Public: Home (event listing), Event details, Checkout, Payment status, Ticket view/download
Attendee: My Tickets, Ticket transfer, Profile
Organizer: Dashboard, Events list, Create/Edit event, Ticket types, Promo codes, Attendees, Staff, Reports, Payouts
Gate Staff: Assigned events, Scanner, Manual search, Scan history
Super Admin: Dashboard, Organizers, Events, Transactions, Refunds, Payouts, Settings, Audit logs

9. Ready-to-paste prompt for Google AI Studio Build mode

Copy everything inside the box below into the Build mode prompt. Large apps work best in stages, so this prompt asks for the core first; use the follow-up prompts in section 10 afterwards.

```
Build a full-stack, mobile-first web application called "TiketiPass" — an event ticke

USER ROLES (role-based access control):
1. Super Admin - approves/suspends organizers, sees all events, sales and commission
2. Event Organizer - creates/edits/publishes/cancels events; defines ticket types (na
3. Gate Staff - logs in on a phone, sees only assigned events, scans QR tickets with
4. Attendee - browses events, buys tickets (guest checkout allowed), views "My Ticke

CORE FEATURES:
```

```
- Public event listing with search and filters (date, category, location) and an ever
- Checkout: choose ticket types and quantities, apply promo code, enter name/email/ph
- Payments: implement M-Pesa Daraja STK Push (sandbox) with a callback endpoint, plus
- Ticket generation: one unique ticket per person with a ticket number and a QR code
- Ticket statuses: ACTIVE, USED, CANCELLED, REFUNDED, TRANSFERRED. Transferring a tic
- Verification: a scanner page using html5-qrcode. The server verifies the signature
- Log every scan (ticket, staff, gate, time, result, device).
- Organizer dashboard with cards and charts: tickets sold per type, revenue, check-in
- Super Admin dashboard: organizers, events, transactions, platform revenue and comm
- Email tickets with a PDF attachment (use a placeholder email service that I can con
- Gemini AI features: (a) "Generate description" button on the create-event form, (b)

FIRESTORE COLLECTIONS: users, organizations, events, ticketTypes, promoCodes, orders

DESIGN: clean, modern, mobile-first, with a dark-mode scanner screen. Use KES currenc

Add seed data: 1 super admin, 2 organizers, 3 events, ticket types, 1 gate staff, and

Include a README with setup steps, all required environment variables (Firebase confi
```

10. Follow-up prompts (send one at a time after the first build works)

1. "Add offline mode to the scanner: download a hashed list of valid tickets for the assigned event, verify offline, queue scans and sync when back online, and flag conflicts."
2. "Add refunds: attendees request a refund, organizers approve or reject it, and approved refunds cancel the tickets automatically."
3. "Add organizer payout requests and Super Admin approval with a payout history."
4. "Add SMS notifications using Africa's Talking for ticket delivery and 24-hour event reminders."
5. "Add rotating QR codes on the My Tickets page that refresh every 30 seconds."
6. "Add a fraud-alert panel that flags tickets scanned at different gates within 60 seconds, and suspicious repeated invalid scans."
7. "Add audit logs for every admin and organizer action, and a page to view and filter them."
8. "Add Firestore security rules so each role can only read and write its own allowed data."
9. "Add rate limiting on login, checkout and scan endpoints, plus input validation on every form."

11. How to download and run the project

1. In Google AI Studio Build mode, open your app and click "Download App" to get the ZIP (or use the GitHub button to push it to a repository).
2. Unzip it on your computer and open the folder in VS Code or Antigravity.
3. Create a `.env` file with the variables listed in the README (Firebase, M-Pesa, QR secret, `GEMINI_API_KEY`).
4. Run:

```
npm install
npm run dev
```

5. For production: deploy to Google Cloud Run (available directly from AI Studio) or to Firebase Hosting.

12. Testing checklist before going live

- [ ] Each role can only see and do what it is allowed to
- [ ] Tickets cannot be oversold when two people buy the last ticket at the same time
- [ ] M-Pesa STK Push works in sandbox and the callback updates the order
- [ ] A ticket scanned twice shows ALREADY USED
- [ ] A forged or edited QR code shows INVALID
- [ ] A transferred ticket's old QR code is rejected
- [ ] A cancelled or refunded ticket is rejected
- [ ] Offline scans sync correctly
- [ ] Emails and PDFs deliver correctly
- [ ] Works on low-end Android phones and slow networks
- [ ] Move M-Pesa from sandbox to production credentials (requires a Safaricom Daraja go-live application)